Sienna Thomas and David Owen
Professor Reichherzer
COP4634
10/19/26

TESTING THE EFFICIENCY OF THREADS USING COLLATZ SEQUENCE

DESCRIPTION

The goal of this project is to test the efficiency of threads using the Collatz Sequence. To do this, we made a program where you can select an upper limit of the range of numbers to process (N), and how many threads to use (T). You can also decide if you want to disable the locking mechanism. The threads will work on computing the sequence for each number up to the upper limit. While this is happening, we are recording how long it takes with chrono. The result shows each stopping time (0-1000) along with the frequency of each one in the results. After this, it shows N, T, and the time it took to compute. We filled 2 csv files with run times, one file for locks, the other for no locks. We made N a constant of 10,000,000 to insure a valuable difference in runtime, and tested each thread count 10 times. The average of these was used for analysis. We also used the frequency of stop times for analysis.

MACHINE DETAILS

Model: Alienware Aurora R9

OS: Ubuntu via WSL on Windows 11

CPU: Intel i7-9700 (8 cores)

RAM: 16GB (2.9GB available)

RESULTS

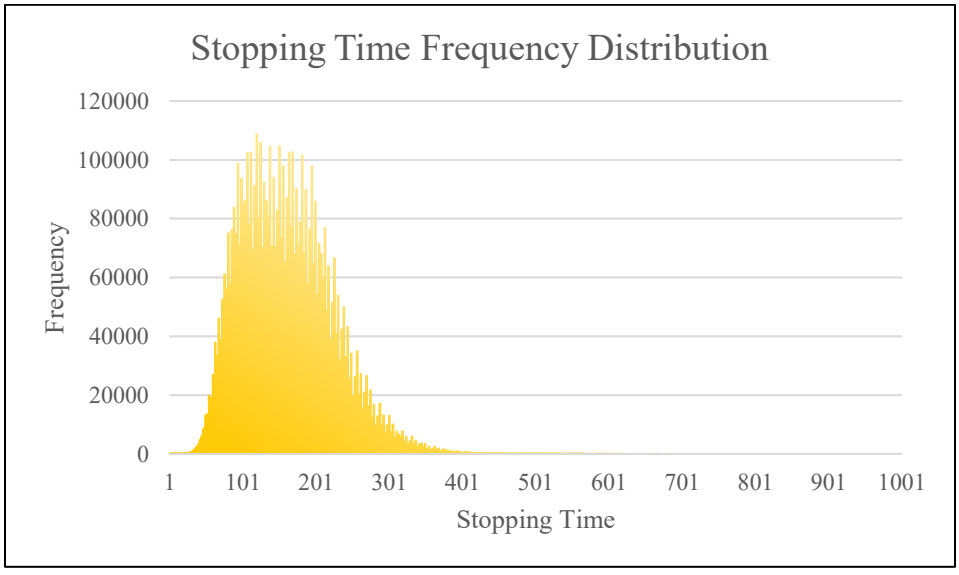


Fig. 1. This shows the frequency for each stopping time (ran with 4 threads, locked).

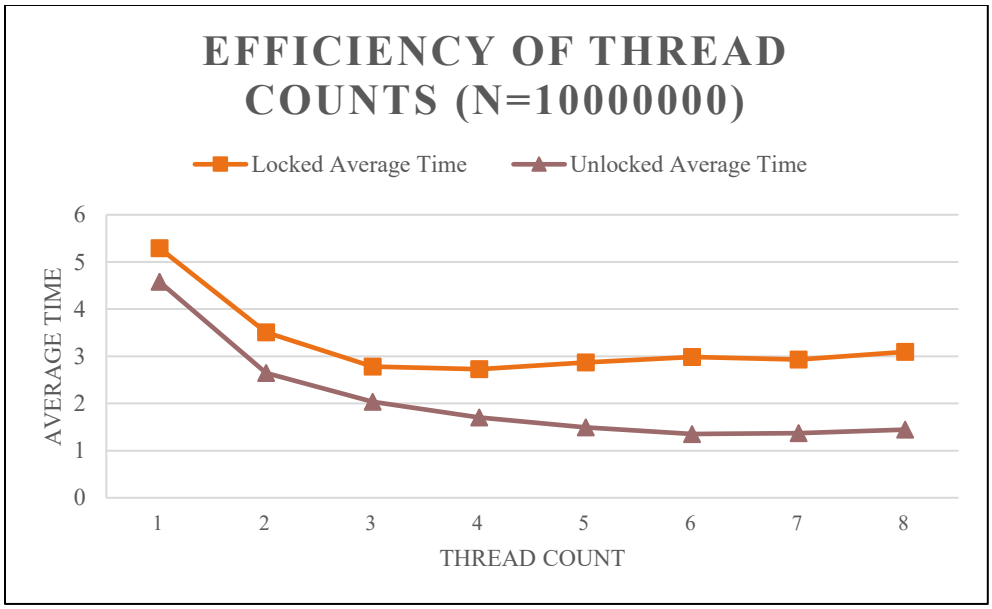


Fig. 2. This shows the average compute time for each thread count (Locked vs Unlocked).

The results from the Stopping Time Frequency show a large of stopping times around 100-200. These are the most common stopping times from the data we tested. There are some outliers to the right of the cluster. If we were to increase N even more, the cluster might widen to include larger numbers. The results from the Efficiency of Thread Counts shows that 1 thread is the slowest for locked and unlocked. Locked is the most optimal at 4 threads before it starts to slow down again. Unlocked is the most optimal at 6 or 7 (haha), but at this point the line straightens almost completely with no change. Unlocked is faster than Locked at every stage.

ANALYSIS

Parallel execution seems to increase performance to an extent. Adding threads will make the program run faster, but there comes a point where too many threads cause too much traffic for the CPU to process. There is an optimal number of threads to get things done, whether you lock it or not. Also, not using locks is faster in every way. It even takes longer to level out on its efficiency, but there is a major flaw with using threads this way. It may look efficient, but it is corrupts data. Since multiple threads can modify a shared variable at any time, data may be overwritten, written 2 times, or not written at all. It is much safer to use locks with threads.

CONCLUSION

This experiment made it very clear how threads work to make programs more efficient. It showed that there can be a limit to how many threads you should use, and it showed that sacrificing compute time to use locks is essential if you want to have uncorrupted data. I was surprised to see the sheer mass of data that threads could work with in such a short time. I learned that the Collatz Sequence is efficient also, with the most common stop times being 100-200 when the upper limit was 10,000,000.